Check for updates

Received: 11 February 2022 | Revised: 15 July 2022 | Accepted: 21 July 2022

DOI: 10.1002/spe.3141

RESEARCH ARTICLE

WILEY

Identification of monolith functionality refactorings for microservices migration

José Correia[1,2] | **António Rito Silva**[1,3]

[1]Distributed Parallel and Secure Systems, INESC-ID, Lisbon, Portugal

[2]Department of Electrical and Computer and Engineering, Instituto Superior Técnico, University of Lisbon, Lisbon, Portugal

[3]Department of Computer Science and Engineering, Instituto Superior Técnico, University of Lisbon, Lisbon, Portugal

Correspondence
António Rito Silva, Rua Alves Redol 9, 1000-029 Lisboa Portugal.
Email: rito.silva@tecnico.ulisboa.pt

Funding information
Fundação para a Ciência e a Tecnologia, Grant/Award Number: UIDB/50021/2020

Abstract

In the process of migrating a monolith to a microservice architecture, functionalities which execute on top of a single database as ACID transactions, have their execution split between several databases, one for each microservice. Therefore, the complexity of functionality migration increases, because of the lack of isolation of the functionality execution, which has to be handled by following an eventual consistency model. On the other hand, the object-oriented approach, commonly followed in the development of monolith systems, promotes fine-grained interactions in the functionalities implementation, which further increases the migration effort due to the large number of remote invocations between microservices, because it increases the number of microservices transactions that have to eventually be consistent. The main contribution of this paper is a heuristic method to help the software architect on the identification of functionality refactorings, to reduce the effort required to migrate the functionality to microservices when applying the *SAGA* pattern in the functionality microservices implementation. The heuristic accuracy and efficiency of the method is evaluated using a dataset extracted from 78 codebases and by comparing the results with expert refactorizations. Additionally, a performance analysis of the heuristic implementation is done.

KEYWORDS

heuristics, microservices, monolith migration, refactoring, SAGA Pattern

1 | INTRODUCTION

The microservices architecture allows the split of a software development project into several small agile cross-functional teams and facilitates independent scalability of the services that constitute the product.[1,2] Therefore, many monoliths have to be migrated to a microservices architecture due to the aforementioned advantages.

To address this problem, there is recent research on the migration of monolith to a microservices architecture, for example,[3-10] which propose techniques to identify candidate decompositions of the monolith. Additionally, they use metrics to assess the quality of the decomposition. Some also allow the architect to interact with a visual representation of the candidate decomposition and even change it while being informed by the recalculation of the quality metrics.[5,7]

The values returned by the quality metrics are strongly dependent on the monolith functionalities structure, because their implementation conforms to the monolith architectural style. However, and surprisingly, this aspect has not been

Abbreviations: ACID, atomicity, consistency, isolation and durability; FMC, functionality migration complexity; SAC, system added complexity.

 © 2022 John Wiley & Sons, Ltd.

addressed by the current monolith migration literature, like as if the migration of a functionality to microservices does not require it to adapt to new architectural styles. As far as we know, only in Almeida and Silva[11] this is addressed. Additionally, the implementation of monolith functionalities in the microservice architecture increases the effort required for their implementation, because it is not possible to guarantee their transactional execution according to strict consistency, as stated by the CAP theorem.[12]

In this paper, we leverage on previous work to support the architect with a mechanism for the refactoring of the monolith functionalities in the context of a candidate decomposition. In,[11] two metrics are proposed to measure the complexity of migrating a monolith functionality to a microservices architecture. These metrics measure the migration effort due to the relaxing of the functionalities ACID transactional behavior[13] into eventual consistency,[14] when they are re-implemented using the *SAGA* pattern[15] in its orchestration style. Additionally, it was observed a code smell for migrations with high complexity. We verified that the high complexity associated with the migration of a functionality is due to the number of inter-candidate microservices invocations, which increases the number of the *SAGAs* intermediate states. The main reason for this situation is that monoliths are implemented using fine-grained object-oriented invocations.

Therefore, although our previous work allowed architects to be informed about the effort required to migrate the functionality, given a candidate decomposition, they are not aware of the possible improvements that occur if fine-grained inter-microservice invocations are refactored into a smaller number of coarse-grained interactions. This situation is worsened because a monolith can have hundreds, if not thousands, of functionalities, making it impractical for a manual inspection of each functionality structure, which indicates that some form of automation would be desirable. Ignoring that the functionalities migration to a microservices architecture requires them to adapt to a different consistency model can mislead the architect on the impact the functionalities structure may have on the migration effort.

The main contribution of this paper is a heuristic method, and its implementation as a refactoring recommendation tool, that, given a candidate decomposition of a monolith, and the monolith set of functionalities, informs the architect about the impact that transforming fine-grained invocations between microservices into coarse-grained ones has on the overall effort of migration. Additionally, the tool recommends, for each refactored functionality, what should be the *SAGAs* microservice orchestration coordinator. This will also help the developer in the refactoring activities because, once the SAGAs orchestrator is identified, it is easier to perform the refactoring steps.

The state of the art on the identification of code smells and design patterns in code follows two distinct approaches: machine learning[16] and heuristics, for example, Reference 17. While the former has shown to be more flexible, it requires extensive manual classification work for training. In this paper we followed the latter, which has shown good results without significant performance penalties. Therefore, a heuristics approach is followed to answer the follow research questions.

Given a candidate decomposition of a monolith, its set of functionalities, their sequences of accesses to domain entities, the data dependencies between those accesses, and metrics for the effort of migrating the functionality to a *SAGA* pattern in the context of the candidate decomposition:

- **RQ1**: Is it possible to recommend the refactoring of a functionality, by merging fine-grained inter-microservice interactions into coarse-grained ones, that minimizes its implementation effort as an orchestrated *SAGA* in the context of the decomposition?
- **RQ2**: Can we characterize the *SAGA* orchestrators that provide a higher reduction to the migration effort?

In the next section we describe the *SAGA* pattern and Mono2Micro approach as the research background, which is followed by the presentation of related work in Section 3. Then, Section 4 presents the solution, Section 5 the implementation, and Section 6 the evaluation. Finally, conclusions are presented in Section 7.

2 | BACKGROUND

2.1 | *SAGA* pattern

The traditional solution for handling data operations in a distributed system spanning multiple services is known as distributed transactions. In this approach, the services provide data-changing resources, and a manager is responsible for invoking each one of the services and managing the global state of the transaction, making sure that the atomicity,

10970024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

consistency, isolation and durability (ACID), properties are achieved.[13] This approach has significant drawbacks when it comes to maintaining data consistency across all services. For instance, if the local transaction in one of the services fails, the manager must make sure that this is reflected in the data state of all the other services. Another drawback occurs when accounting for service availability, because for a distributed transaction to be successful all the services must be available, which creates tight coupling between all the participants.

The so-called gray literature proposes the use of the *SAGA* pattern to implement business functionalities in the microservices architecture[15] in order to maintain data consistency across the multiple services and without using atomic distributed transactions. This pattern is based on the seminal work by Hector-Molina and Kenneth[18] and addresses the lack of isolation due to the creation of intermediate states that are visible outside the scope of the functionality execution. These intermediate states result in the introduction of eventual consistency, which adds to the complexity of the functionality, because the business logic becomes intertwined with the management of the intermediate states, for instance, the need to have compensating transactions.

Each local transaction updates data within a single service using an ACID transaction and then publishes a message or event to trigger the next local transaction in the *SAGA*. If a local transaction fails, because it violates a business rule, then the *SAGA* executes a series of compensating transactions that undo the changes that were made.

Richardson[15] describes two different strategies to implement *SAGA*s: orchestrations and choreography. In an orchestration the business functionality logic is centralized in a coordinator component, an orchestrator, that informs *SAGA* participants about which local transactions should be executed. The orchestrated microservice performs the local transaction and reports back to the orchestrator, informing if it was successfully completed or not. This communication can be done by any regular HTTP implementation or by an asynchronous event-producer and event-consumer design. In a choreography there is no central entity that coordinates the execution of the business functionality logic. Instead of relying on an orchestrator, the *SAGA* is triggered by an event published to a message broker. *SAGA* participants, configured to consume that event, will then trigger their local transactions, and upon completion, publish an event back to the message broker, which will trigger local transactions in other participants. These interactions go on until all the required local transactions in the *SAGA* are completed or until one fails and a compensating transaction is initiated.

2.2 | Mono2Micro system

The Mono2Micro[*] is a software system that implements tools to support software architects on the task of migrating codebases from a monolithic to a microservices architecture. Mono2Micro is the test bed of previous work,[7,11,19] that minimizes the number of distributed transactions necessary to migrate a monolith functionality to a microservices architecture. The functionality refactoring heuristics is implemented on top of Mono2Micro candidate decompositions.

In Figure 1 are presented the main features implemented by the Mono2Micro system:

1. Data extraction from the source code of codebases implemented using Spring-Boot, by applying static or dynamic analysis algorithms.[20] This analysis generates a call-graph containing the order of entity accesses and type of access (read or write) for each functionality of the codebase. A functionality is represented by a Spring-Boot controller, which implements a monolith webservice;
2. Clusterization of the domain entities using a hierarchical clustering algorithm[†] implemented by Scipy Python library, which generates a dendrogram.[7] The similarity measures used by the clustering algorithm are defined such that domain entities accessed by the same functionalities should be in the same cluster, to minimize the number of local transactions necessary to implement a functionality;
3. Manually cutting the dendrogram into a set of different clusters, to create a decomposition. The architect can do this by choosing one of two distinct methods:
   - Choosing the total number of clusters that the cut must have.
   - Choosing the maximum distance between domain entities inside each cluster.
4. Visualizing complexity metrics that enable the architect to assess the quality of the decomposition created.[19]
5. Experimenting with the clusters by allowing the architect to rename, merge and split clusters, as well as move data entities between them.

[*] https://github.com/socialsoftware/mono2micro.

[†] https://docs.scipy.org/doc/scipy/reference/cluster.hierarchy.html.

1097024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

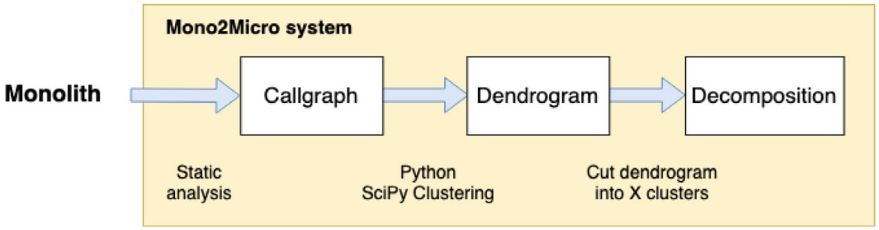


FIGURE 1 Mono2Micro approach pipeline

6. Refactoring, manually, any functionality of the codebase by applying a set of refactorization operations, including: Add Compensating, Sequence Change, Local Transaction Merge and Define Coarse-Grained Interactions.[11]

3 | RELATED WORK

There is extensive research on the migration of monoliths to the microservices architecture, for example References 3-10, but the focus has been on service identification[21,22] while in this paper we address the problem of the identification of functionality refactorings to reduce the migration effort, given a particular service identification.

On other hand there are work on service granularity, and the ability to change it, either statically or dynamically. Nakazawa et al.[5] propose a set of operations which change a candidate decomposition of a monolithic system, such as moving an entity between clusters or merging them, while recalculating the amount of inter-cluster communication. However, they do not address the refactoring of the monolith functionalities in the context of a decomposition. On a dynamic approach, Hassan et al.[23] propose the dynamic adaptation of microservices granularity through the merging and decomposition of microservices based on an architectural added value. In this paper we do not change the decomposition but identify refactoring functionalities toward a smaller migration effort.

The approaches on service identification and on service granularity built on top of microservices quality metrics, such as cohesion, coupling and complexity,[24] or scalability.[25] This paper builds on previous work, Almeida and Silva,[11] that proposes two metrics to measure the complexity associated with the functionality migration to *SAGA*s pattern in a microservices architecture and the complexity that this implementation adds to the implementation of other functionalities. These metrics are built on the number of intermediate states and the number of inter-microservice invocations associated with the functionality implementation. Additionally, Almeida and Silva,[11] proposed three operations to refactor a functionality, given a candidate microservices decomposition, where the refactoring of fine-grained invocations into coarse-grained ones shown to have a significant impact on the reduction of complexity of migrating a functionality in a distributed context. However, this process is not automated, and requires a large cognitive effort from the architect in the identification of which refactorings minimize complexity.

The current research on identifying code-smell and design patterns follow two main trends: heuristic and machine learning classification. In the heuristic classification approach, a set of code metrics is computed and combined to create detection rules.[17,26-28] However, some drawbacks have been identified in this approach due to the low agreement between different detectors and difficulties in finding suitable thresholds to be used for detection.[29] Fontana et al.,[16,30,31] propose the use of machine learning techniques. These techniques tend to be more flexible and independent compared to heuristic classification, since learning by example allows for better handling of distinct scenarios. However, they require extensive manual classification work to train the machine learning algorithm. None of the mentioned research addresses the identification of fine-grained invocations, which is a code-smell in a distributed system, neither their refactoring into coarse-grained invocations.

The research on the migration of monolith systems to a microservices architecture that uses automatic and semiautomatic methods already use heuristic, for example, Gyselet al.[3] and Jin et al.,[6] and machine learning, for example, Ma et al.[32] and Abdullah et al.,[33] techniques to the identification of candidate decompositions, but there is no work on the automatic identification of code-smells that recommends refactorings to ease monolith functionality migration in the context of a candidate decomposition.

1097024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

In this research we refactor monolith business functionalities to minimize the migration effort required to implement them as *SAGA*s in the microservices context. Orchestration is the chosen strategy for the *SAGA*s implementation, because its stepwise execution of the local transactions reduces the implementation complexity, as suggested by Richardson.[15]

4 | SOLUTION

Figure 2 presents the approach of refactoring monolith functionalities into *SAGAs*. The monolith is defined as a triple of the monolith functionalities, its domain entities and their traces of accesses to domain entities. This model of the monolith can be obtained either by static analysis of its code, or dynamic collection of its execution traces. In the monolith, each functionality is executed in a transactional context that is supported by a single database transaction. The functionality execution is modeled by its sequence of accesses to the domain entities. Domain entities represent the tables in the database. Two relations are relevant in the execution traces, the sequence relation, which defines the accesses order, and the data relation, which defines data, consumer/producer, dependencies between domain entities accesses.

As an example, consider a monolith, $F = \{f\}$, $E = \{e_1, e_2, e_3\}$, $T = (\{(e_1, r), (e_2, w), (e_3, r)\}, \{< (e_1, r), (e_2, w) >, < (e_2, w), (e_3, r) >\}, \{< (e_1, r), (e_2, w) >\})$. It has a single functionality (f), 3 domain entities (e_1, e_2, e_3), and a trace that does 3 accesses ($(e_1, r), (e_2, w), (e_3, r)$), 2 reads and 1 write, where e_1 occurs before e_2 ($< (e_1, r), (e_2, w) >$), e_2 before e_3 ($< (e_2, w), (e_3, r) >$), and the write in e_2 consumes data that is produced when e_1 was read ($< (e_1, r), (e_2, w) >$). The execution of this functionality is supported by an ACID transaction of the monolith database.

Given the monolith, a candidate decomposition is a partition of its domain entities, which represent the different databases each microservice will implement. The functionalities traces are also partitioned, where a single ACID monolith transaction is split into several ACID local transactions, plus the remote invocations between them.

As an example, C is a candidate decomposition that has two clusters, $C = \{\{e_1, e_2\}, \{e_3\}\}$, and the partitioning of the functionality results in two local transactions, respectively, sequences $< (e_1, r), (e_2, w) >$ and $< (e_3, r) >$, and a remote invocation $< (e_2, w), (e_3, r) >$.

Finally, associated with the candidate decomposition, we calculate its complexity, which represents the cost of implementing each functionality as a distributed transaction. This cost increases with the number of local transactions, plus the reads and writes that other functionalities do on entities accessed by the functionality. Due to the lack of ACID transactional behavior in the execution of the functionality, writes generate intermediate states, which, in an eventual consistency transactional context, increase the complexity of its implementation: the functionality has to consider all possible intermediate states.

For instance, suppose that the monolith has another functionality, f' that writes e_1 and reads e_2, and this functionality also needs to be implemented as a distributed transaction. The implementation of f, when reading e_1 has to consider the

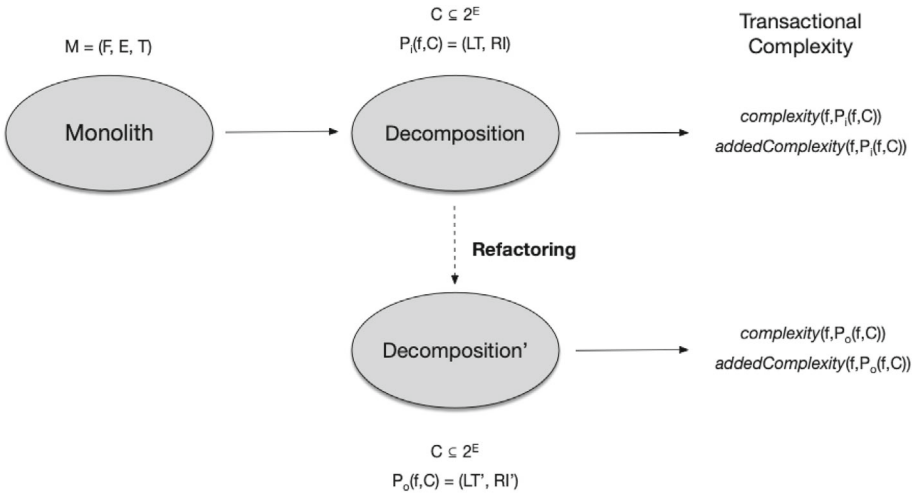


FIGURE 2 Functionality *SAGA* refactoring approach

10970024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

possible intermediate states set by f', whereas the implementation of f' has to consider the possible intermediate states of e_2, set by f. The former is f functionality complexity, whereas the latter is the complexity f adds to other functionalities.

In this paper, we present an algorithm that reduces the complexity of implementing the candidate decomposition by refactoring its functionalities into *SAGAs* ($P_i(f, C)$ to $P_o(f, C)$). After the refactoring, the new candidate decomposition has the same clusters, but the functionalities have a smaller number of local transactions and produce less intermediate states. Additionally, for each refactoring, it is identified which cluster can play the role of *SAGA* orchestrator.

In the following, we present a formalization of concepts related to the functionality migration from a monolith architecture to a microservices architecture, applying the *SAGA* pattern, which also includes the complexity metrics that will be minimized by the solution. Then, is presented the algorithm responsible for the recommendation of *SAGA* orchestrations, in the context of a monolith decomposition and functionality traces.

4.1 | Functionality migration

Typically, the execution of a monolith functionality is triggered by an interaction between an external entity and the monolith, for instance, an end user. During this interaction, a set of accesses, read or write, to the persistent domain entities occur in a consistent transactional context. These accesses can be collected as traces, which represent how the functionality interacts with the persistent entities. This information is relevant from the point of view of the functionality transactional behavior. Note that a functionality trace can have several sequences of accesses, because of the internal conditions of execution, due to the internal state of the monolith, or due to the functionality input parameters. All the sequences in a functionality trace have a common prefix of accesses, which can be empty for executions of a functionality that diverge in the first access.

Therefore, the migration of a monolith functionality is based on the information collected on the functionalities accesses to the monolith domain entities, the sequences of accesses, and their data dependencies. More precisely, a monolith is defined as a triple (F, E, T), where F denotes its set of functionalities, E the set of domain entities, T a set of traces of the monolith functionalities accesses.

The traces are defined as a triple (A, S, D), where $A = E \times M$ is a set of read and write accesses to domain entities ($M = \{r, w\}$), $S = A \times A$ the execution sequence relation between elements of A, which indicates that the first element of the pair was invoked, in the context of a monolith functionality, immediately before the second element, and $D = A \times A$ the data dependencies between accesses in the context of a sequence, where the first element is a read access, the producer, and the second element a write access, the consumer. Given a sequence, $s \in S$, its transitive closure, s_t, is a total order, in particular, any element is comparable and there is no circularities, $\forall_{(a_i,a_j)\in s_t}(a_j, a_i) \notin s_t$. This property is important to guarantee that a monolith functionality is represented by sequences of accesses where given two accesses in a sequence one occurs before the other. For each functionality $f \in F$, $f.s \subseteq S$ denotes its set of access sequences. Additionally, the data dependencies occur in the context of a functionality access sequence and conform to the sequence order, $\forall_{(a_i,a_j)\in D}\exists_{s\in f.s}(a_i, a_j) \in s_t$.

As an example, a sequence of a functionality f, $s \in f.s$, $s = \{<(e_1, r), (e_2, w)>, <(e_2, w), (e_3, r)>\}$. Its transitive closure $s_t = \{<(e_1, r), (e_2, w)>, <(e_2, w), (e_3, r)>, <(e_1, r), (e_3, r)>\}$ defines a total order. The data dependency $<(e_1, r), (e_2, w)>$ states that the value of e_1 is used in the write of e_2, and it conforms to the sequence relation. Another sequence $s' \in f$ can be a prefix of s, $s' = \{<(e_1, r), (e_2, w)>\}$.

Note that, while the execution sequence relation is necessary to understand the functionality transactional behavior, the data dependencies are relevant for the functionality refactoring, because when the order of accesses is changed it is necessary to preserve the data dependencies. For instance, if a write access uses a value previously obtained in a read access, it is not possible the change their order of execution, the write cannot occur before the read.

The functionality migration occurs in the context of a candidate decomposition, which is a partition of the domain entities. Each candidate microservice is represented by a cluster of domain entities. In the context of the decomposition, a functionality accesses the different clusters, possibly accessing the same cluster more than once, and each cluster access occurs as a local transaction, while the accesses between cluster correspond to remote invocations.

Therefore, given the set of sequences of accesses $f.s$ of a functionality f and a decomposition into a set of clusters of the domain entities, $C \subseteq 2^E$, where the clusters are nonempty and a domain entity is in exactly one cluster, the partition

1097024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

of a sequence s of a functionality f, $s \in f.s$, $P(s, C) = (LT, RI)$ is defined by a set of local transactions LT and a set of remote invocations RI, where each local transaction:

1. is a tuple (A, S)
2. is a subsequence of the functionality sequence of accesses, $\forall_{lt \in LT} : lt.s \subseteq s$;
3. contains only accesses to the domain entities of a single cluster, $\forall_{lt \in LT} \exists_{c \in C} : lt.a.e \subseteq c$;
4. contains all consecutive accesses in the same cluster, $\forall_{a_i \in lt.a, a_j \in s.a} : ((a_i.e.c = a_j.e.c \wedge (a_i, a_j) \in s) \Rightarrow (a_i, a_j) \in lt.s) \vee ((a_i.e.c = a_j.e.c \wedge (a_j, a_i) \in s) \Rightarrow (a_j, a_i) \in lt.s)$;

From the definition of local transaction, results the definition of remote invocations, which are the elements in the sequence that belong to different clusters, $RI = \{(a_i, a_j) \in s : a_i.e.c \neq a_j.e.c\}$.

Note that, in these definitions, we use the dot notation to refer to elements of a composite or one of its properties, for example, in $a_j.e.c$, $.e$ denotes the domain entity in the access, and $.c$ the cluster the domain entity belongs to. In another example, $lt.a$ denotes the set of accesses in the local transaction lt.

Once we have partitioned all functionality access sequences according to the decomposition, it is necessary to define the set of local transaction and remote invocations of the functionality such that the repeated elements, due to the common prefixes in the sequences, are not considered. Therefore, a partition of a functionality f, given a decomposition C, is the union of the partition of each one of its sequences, $P(f, C) = \cup_{s \in f.s} P(s, C)$, where local transactions and remote invocations that are in the common prefixes of sequences are not repeated. Additionally, sequence and data dependence relations between local transactions are inferred from the functionality original sequence and data dependence relations, and are denoted by $<_S$ and $<_D$, respectively. These two relations will be used in the refactoring algorithm.

For instance, considering the previously referred functionality f and a decomposition where e_1, e_2 belong to a cluster and e_3 to a different one, we get two local transactions, lt_1, lt_2, and a remote invocation, ri, associated with the functionality. The local transaction lt_1 is $(\{(e_1, r), (e_2, w)\}, \{< (e_1, r), (e_2, w) >\})$, local transaction lt_2 is $(\{(e_3, r)\}, \{\})$, and $\{< (e_2, w), (e_3, r) >\}$ is the remote invocation. Finally, $lt_1 <_S lt_2$ and $<_D$ is empty, because the only data dependency occurs inside a local transaction.

The partition of a functionality f, given a decomposition C, and obtained using the above rules, is called the initial partition of f and it is denoted by $P_i(s, C)$ and $P_i(f, C)$, for, respectively, a sequence s and all sequences of f.

The refactor of a functionality, given a decomposition, is done transforming its initial partition into a partition that represents a *SAGA* implementing an orchestration, where the orchestrator is a pivot of the interactions between local transactions. Therefore, a partition of a functionality sequence of accesses is an orchestration, if there is a cluster that is involved in all remote invocations. This cluster, c, is an orchestrator if $\forall_{(a_i, a_j) \in RI} : a_i.e.c = c \vee a_j.e.c = c$. Note that, by definition, a remote invocation occurs between different clusters. Using the previous example, the initial partition is also an orchestration, it has a single remote invocation, and the cluster that contains e_1, e_2 can be considered as the orchestrator.

The refactor of a functionality sequence of accesses is the transformation of its initial partition into a partition that is an orchestration. This refactoring is done to minimize its migration complexity in the context of a distributed transaction. This complexity is measured by the number of local transactions and the impact that a local transaction has on the other functionalities migration. This is due to the lack of isolation that each local transaction brings to the functionality migration in a decomposition, which requires the introduction of compensating transactions and the need to handle the possible intermediate states of the domain entities accessed by different functionalities.[11,19]

Definition 1 (Functionality migration complexity (FMC)). The complexity of migrating a functionality f, given its partition (LT, RI) in a decomposition, is the sum of the complexity of each one of its local transaction:

$$complexity(f, (LT, RI)) = \sum_{lt \in LT} complexity(lt)$$

Definition 2 (Local transaction complexity). The complexity of a local transaction depends on the domain entities it writes, because it is necessary to implement compensating transactions for when the functionality execution has to rollback. It also depends on the domain entities it reads, because it is necessary to consider the intermediate states that

10970024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

other functionalities introduce when they write them in their local transactions. The latter considers each functionality only once, because the complexity is associated with the cognitive effort required to analyse the functionality intermediate states.

$$complexity(lt) = \#writes(\{lt.s\}) + \sum_{e \in reads(\{lt.s\})} \#\{f_i \in F \setminus \{f\} : \exists_{lt_j \in P(f_i,C)} e \in writes(lt_j.s)\}$$

where

$$writes(s) = \{e : \exists_{s_i \in s}(e, w) \in prune(s_i)\}$$

$$reads(s) = \{e : \exists_{s_i \in s}(e, r) \in prune(s_i)\}$$

and the prune function, when applied to a sequence of accesses, removes all read accesses to a domain entity after the first read access to that entity, and removes all accesses to an entity after the first write access to that entity. This function identifies which accesses are relevant for other functionalities, because the local transaction executes with strict consistency properties. Therefore, after a write, local reads do not need to concern about external writes done by other functionalities, and only the first read has to consider the possible intermediate states generated by other functionalities.

For instance, $prune(< (e_1, w), (e_2, w), (e_1, w) >) =< (e_1, w), (e_2, w) >$ because only one write is visible to other functionalities, due to the isolation of the local transaction. On the other hand, $prune(< (e_1, w), (e_2, w), (e_1, r) >) =< (e_1, w), (e_2, w) >$ because the local read access reads the value set inside the local transaction, and so it does not depend on other functionalities write accesses to e_1.

Definition 3 (System added complexity (SAC)). The migration of a functionality impacts the other functionalities migration complexity. For instance, if a write is done on an entity e due to the execution of a functionality f_i then every other functionality f_j (where $i \neq j$) that reads the same entity e must have to be changed to handle the possible intermediate states. Hence, the effort of migrating f_j depends on the number of writes done by f_i in entities that f_j reads.

$$addedComplexity(f_i, (LT, RI)) = \sum_{lt \in P(f_i,C)} \sum_{f_j \in F \setminus \{f_i\}} \#(\{e : \exists_{lt_j \in P(f_j,C)} e \in reads(lt_j.s)\} \cap writes(lt.s))$$

The SAC metric is redundant in terms of the overall complexity of migrating the monolith, but is captures the complexity that each functionality adds to the others functionalities migration. This information helps to identify the functionalities whose refactoring may have a higher impact on the reduction of the system complexity.

4.2 | SAGA refactorization algorithm

Figure 3 presents the global structure of the refactoring algorithm. It uses a brute-force strategy, where each cluster participating in the functionality is considered as a candidate orchestrator. Then, it rewrites the functionality to become a *SAGA* implementation by introducing empty local transactions. Finally, it tries to merge local transactions that have the same caller and called clusters, that is, converts fine-grained invocations into coarse-grained ones. The merge step is constrained by the existing data dependencies.

The main algorithm $estimateSagas(F, C)$, Listing 1, uses the functionalities initial partitions P_i, given a candidate decomposition, and calculates, for each functionality, what is the complexity migration values when considering each of the clusters that are involved in the functionality implementation as orchestrators. The result is an ordered array of migration complexities where the cluster that has the lowest value is the recommended orchestrator for the functionality refactoring. The algorithm has three steps: (1) sets a cluster orchestrator of the functionality, $setOrquestrator(P_i(s, C), c)$; (2) merges the invocations between the same pairs of clusters, to obtain coarse-grained invocations, $mergeInvocations(p, c)$; (3) calculates the complexity, $complexity(f, p_c)$.

10970244s, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

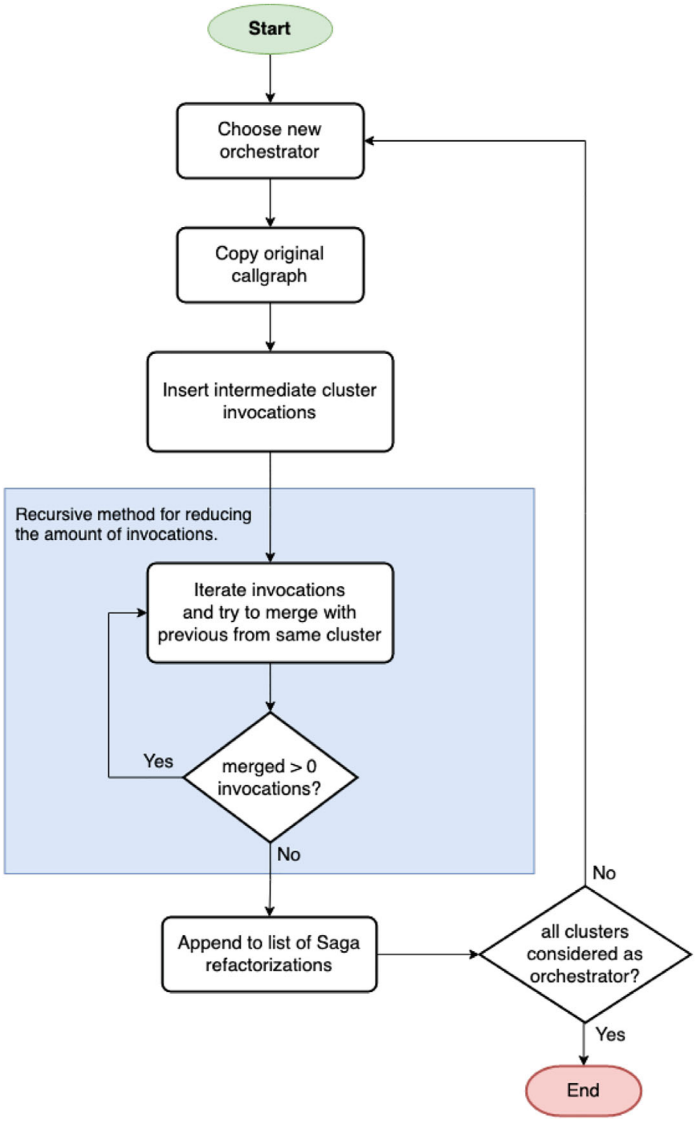


FIGURE 3 Refactoring algorithm structure

```
estimateSagas(F,C) {
   complexities := array[F.size, C.size]
   for f := range F {
      for c := range C {
         p_c := {}
         for s := range f.s {
            p := setOrquestrator(P_i(s,C), c)
            p := mergeInvocations(p, c)
            p_c := p_c ∪ p
         }
         complexities[f,c] := complexity(f, p_c)
      }
   }
   complexities[f].sorted
}
```

Listing 1: *SAGA* estimator

10970024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

To set a cluster as the orchestrator of an initial partition of a functionality, it is necessary to add empty local transactions to the orchestrator cluster and remote invocations to the other clusters, Listing 2. The execution sequence of the local transactions is preserved, because the only change is the introduction of the empty local transactions, which work as pivots between the invocations. Therefore, the data dependencies between the local transactions are not changed.

```
setOrchestrator((LT,RI),c) {
   resultLT := LT
   resultRI := {}
   skip := false
   for lt = range LI.sortedBy(<S) {
      if (!skip) {
         if (lt.c == c) {
            resultRI := resultRI ∪ {(lt.previous,lt), (lt,lt.next}
            skip := true
         } else {
         emptyLT := new emptyLT(c)
            resultLT := resultLT ∪ {emptyLT}
            resultRI := resultRI ∪ {(lt.previous,emptyLT), (emptyLT, lt)}
            skip := false
         }
      }
   }
   return (resultLT, resultRI)
}
```

Listing 2: Set Orchestrator for Partition

Figure 4 exemplifies a transformation made according to *setOrchestrator* in Listing 2, where we set the empty pivot orchestrator invocations between each one of the other cluster invocations, whenever necessary. A is the chosen orchestrator cluster.

The merge of invocations, Listing 3, merges the invocations that occur between the orchestrator and another cluster, if there is no data dependence with a local transaction that occurs in between. It repeats while two remote invocations can be merged into a coarse-grained remote invocation. The `canMerge` condition, which applies to pairs of invocations between the same clusters, is defined by the data dependence relation $<_D$, such that, it does not exist a local transaction between the two remote invocations that the local transactions in the second remote invocation have a data dependence on. As a result, the merged local transaction sequences are concatenated (pruned) and the data dependencies between them are preserved. $RI.after(ri_1)$ denotes all the remote invocations that occur after ri_1.

```
mergeInvocation((LT,RI),c) {
   resultLT := LT
   resultRI := RI
   while (ri1 in range RI, ri2 in range RI.after(ri1), canMerge(ri1,ri2)) {
      ri1.caller.prune(ri2.caller)
      ri1.callee.prune(ri2.callee)
      resultLT := resultLT \ {r21.caller, ri2.callee}
      resultRI := resultRI ∪ {(ri2.previous.caller,ri2.next.callee)} \ {ri2.previous, ri2, ri2.next}
   }
   return (resultLT, resultRI)
}
```

Listing 3: Refactor through merge of fine-grained invocations into coarse-grained

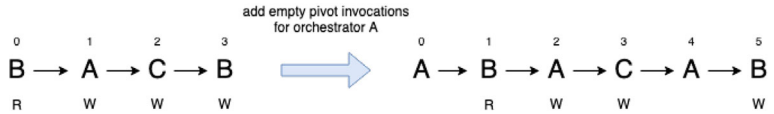


FIGURE 4 Operation of adding pivot orchestrator invocations

10970244, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

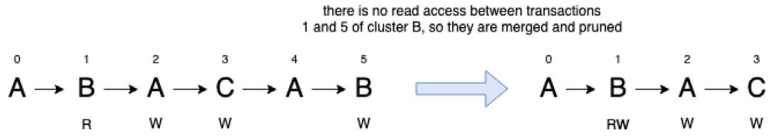


FIGURE 5 Operation of merging and pruning invocations without data dependence

The $canMerge(ri_1, ri_2)$ function can be parameterized to allow variations on the scope (δ) of data dependencies to be considered. When $\delta = 1$ a data dependence is only considered if it occurred in the local transaction immediately before the local transaction being analyzed, however, if $\delta = \infty$ are considered all the local transactions in between the ones to be merged. Allowing different scopes permits different evaluations because a previous read does not necessarily imply that it is used in a subsequent write.

Figure 5 illustrates a transformation performed by *mergeInvocation* in Listing 3, where all invocation that do not have data dependencies, according to the value of δ, are merged. Note that, in the example the prune of the access sequences is equal to the sequences concatenation, because the read access occurs before the write access, considering that they are accesses to the same domain entity.

Note that, if in Figure 5 entity *C* has a read access instead of a write, the refactoring could be done for $\delta = 1$, but not for $\delta \geq 2$.

5 | IMPLEMENTATION

The implementation of the refactoring algorithm is done on top of the Mono2Micro system, see Figure 2. The MVC controllers callgraphs correspond to the functionality traces, and are obtained through static or dynamic analysis. Given a candidate decomposition, which is a partition of domain entities into clusters, the refactoring applies to the monolith functionalities traces.

The refactorization tool is implemented as a Golang[‡] service [§]. Given the sequence of domain entity accesses for each functionality and a decomposition of the domain entities into several clusters, where each cluster represents a microservice, it applies the heuristic algorithm to each one of the functionalities. Therefore, it indicates, for each functionality, the orchestration *SAGA* that has lower complexity, supporting the software architect on the identification of the changes to be made in each one of the functionalities. Prior to this tool, the manual refactoring was error prone, if not completely impossible to be handled by a human. Note that, some the of initial partitions of the functionalities can have hundreds of local transactions.

The tool graphical interface is included in the Mono2Micro system frontend application, and it follows a set of user interaction requirements:

- The user must have some context of how the tool works;
- The user must be able to request the refactorization of a codebase and comeback later to check its status;
- The user must be able to see the metrics of the functionalities initial partition and compare them with the metrics of the *SAGA* refactoring;
- The user must be able to see the refactored traces which show what are the changes to make to the controller's interactions.

In order to fulfill these requirements, a new static page was created, shown in Figure 6. This interface is built using the ReactJS framework and communicates directly with the Refactorization Tool. In this page, the user is introduced to the service and the configuration of the data dependence threshold, before the refactorization starts. After the refactorization is requested, the user can leave the page and comeback after, as the processing is done asynchronously.

[‡]https://golang.org/doc/.

[§]https://github.com/socialsoftware/mono2micro/tree/master/tools/functionality_refactor/src.

1097024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

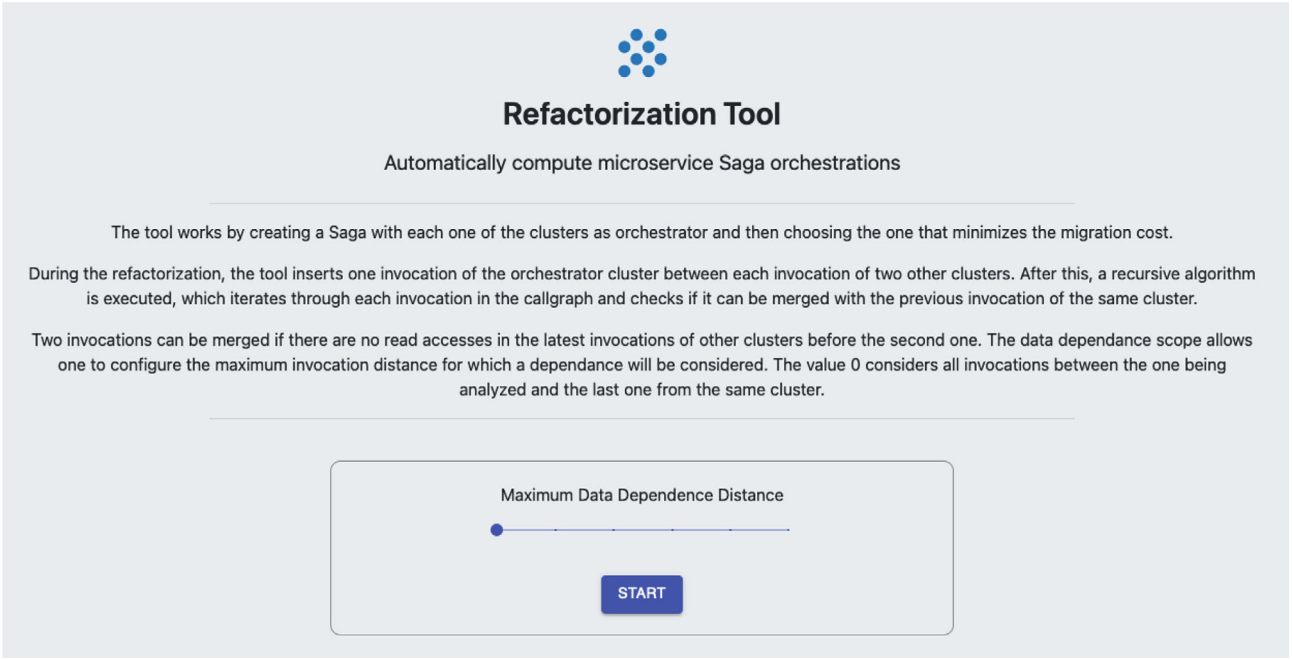


FIGURE 6 Mono2Micro dashboard that is used to interact with the refactorization tool

UPDATE TABLE

Results of last estimation for the valid Saga controllers:

Controller	Status	Orch.	FRC	SAC	Invocations	Accesses	FRC Reduction	SAC Reduction	Merges
AdminController.deleteAllFragments	COMPLETED	0	570	1649	11	62	376	1312	39
AdminController.deleteFragment	COMPLETED	0	593	1636	12	63	369	1355	39
AdminController.deleteVirtualEdition	COMPLETED	1	946	2536	10	83	943	3187	34
AdminController.loadVirtualCorpus	COMPLETED	3	1198	3155	12	100	2004	4513	79
AdminController.removeTweets	COMPLETED	3	85	393	6	19	12	-2	3
VirtualEditionController.deleteVirtualEdition	COMPLETED	3	974	2394	8	75	977	3394	36

FIGURE 7 Dynamic table of results were each row corresponds to a functionality being refactored

Once the functionalities refactorings are completed their status is updated in the table shown in Figure 7, where the user visualizes the final complexity metrics and the complexity reductions resulting from the refactoring.

When the user selects a functionality, a drop-down panel appears, showing (Figure 8) the *SAGA* orchestration that minimizes the migration cost. Additionally, the sequence of cluster invocations, and the domain entity accesses in each one of them are presented, which informs the architect about what should be the result of the refactoring. In the figure, cluster number 3 is the orchestrator.

6 | EVALUATION

To answer the first research question (RQ1), and validate that refactorings can minimize the effort associated with the migration, we executed the tool for a dataset of 78 codebases and analyzed the results to evaluate the reduction of complexity. To validate the accuracy of the refactorizations, we compared with the refactorizations done by an expert[11] on the LdoD codebase, in terms of complexity reduction and feasibility.

1097024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

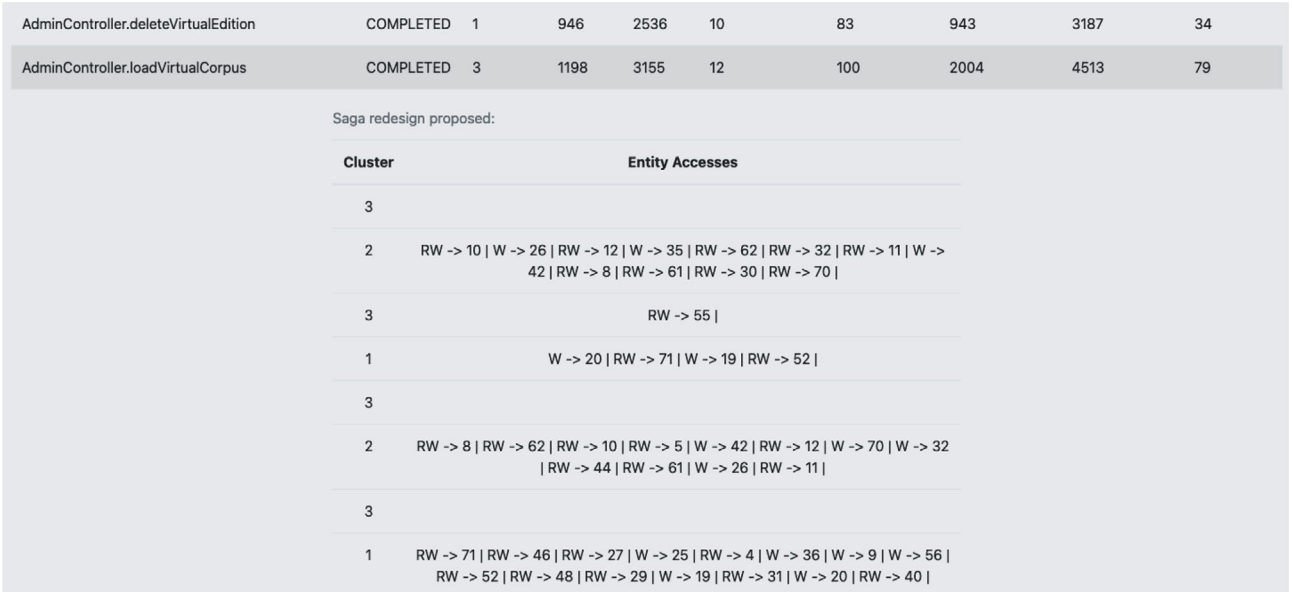


FIGURE 8 Refactorization dropdown which shows the proposed Saga

6.1 | Complexity reduction analysis

The 78 codebases[¶] are monolith systems implemented using Spring-Boot and an object-relational mapper (ORM), where Java Persitence API (JPA) is used in 75 of the codebases and Fénix Framework[#] in the other 3. The Spring-Boot controllers are used to implement the monolith transactional functionalities that access the persistent domain entities implemented using the ORM. Thus, the monolith functionalities are captured by analyzing the Spring-Boot controllers and the analysis of the ORM gives the domain entities accesses. The data set was generated through static analysis of the source code, where sequences of read/write accesses to the domain entities are obtained for the functionalities.

Each codebase was decomposed into clusters, and, after applying the initial partition to the functionalities sequences, the functionalities were selected based on two conditions: (1) the sequence of accesses has at least one write access; (2) the sequence of accesses includes more than two local clusters. These are pre-conditions for the implementation of the functionalities as *SAGA*s, by excluding queries and functionalities that cannot be implemented as *SAGA*s. After these selections, the dataset was formed by 652 functionalities.

Table 1 presents the results, on average, of applying the refactoring heuristic to the 652 functionalities. It compares the initial complexity with the complexity of the refactored functionalities. The heuristic was applied for three variations of the data dependencies between local transactions, where $\delta = 1$ means that there is no data dependence with the local transaction that executed immediately before, $\delta = 2$ means that there no data dependence with the two local transactions that executed immediately before, and $\delta = \infty$ that there is no data dependence with any local transaction that executes in between the remote invocations to be merged. It is necessary to analyze these cases because the data dependencies are obtained from an analysis of the sequences and this may not necessarily mean that a local transaction effectively uses the data read in a previous local transactions. For instance, if there is a write access in a local transaction we consider that data dependencies exist with all the local transactions that occur before and that have at least one read access, because the value read may be used to calculate the value that is going to be written. This is a limitation of the static analysis used to collect the monolith behavior, which does not capture the data-flow, and so we considered the worse situation in terms of data dependencies.

The table also presents the percentage of reduction in the number of local transactions and domain entities access. Additionally, it includes the average number of merges, and the heuristic performance, which was calculated running 10 samples on an Intel i5 2.90 GHz, 2 Cores with 4 threads each.

[¶]https://github.com/socialsoftware/mono2micro/tree/master/tools/functionality_refactor/codebases.
[#]https://fenix-framework.github.io/.

10970244, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 Average values for running the refactoring heuristic for 652 functionalities, considering $\delta = 1, 2, \infty$

		Final			% Reduction		
Metric	**Initial**	$\delta = 1$	$\delta = 2$	$\delta = \infty$	$\delta = 1$	$\delta = 2$	$\delta = \infty$
FMC	329.0	78.4	86.7	95.6	76.1%	73.6%	70.9%
SAC	404.9	150.7	170.6	201.3	62.7%	57.9%	50.3%
Local transactions	35.5	4.3	4.7	5.1	87.9%	86.8%	85.6%
Entity accesses	38.7	10.3	11.3	12.4	73.4%	70.8%	67.9%
Number of merges		22.5	21.7	21.4			
Execution time (sec)		3.76	4.13	4.40			

From the observation of the table can be concluded that there is a significant positive impact of the refactorings on the complexity reduction and that the scope of data dependencies also impacts on final complexity. As expected, a larger scope will reduce the number of possible merges, but it is not significant, which means that in most of the functionalities there is a data dependence between a local transaction and the local transaction that executes immediately before. It is also worth mentioning the reduction in the number of domain entities access, which results from the fact the accesses are localized inside the same local transactions and, so, the visibility of their effects to other local transactions is reduced.

The analysis of the codebases allows us to partially answer the first research question (RQ1), due to the reduction in the complexity, but it is necessary to analyze the accuracy of the results: are the recommended refactorings feasible?

6.2 | Refactorization accuracy analysis

The heuristic refactoring method was applied to the LdoD archive[‖], which consists of 133 controllers and 67 domain entities. The decomposition used was done by an expert which cuts the system into 5 clusters. The analysis involved comparing the results with the refactorings of 9 controllers of LdoD,[11] where the expert refactored the code and calculated the initial and final FMC and SAC. We do not compare the SAC complexity because it depends on the type of local transaction, which the algorithm cannot predict. For instance, whether a local transaction is compensatable or not. The heuristic always considers the worst-case scenario.

Table 2 presents the results when the heuristic was applied with a data dependence scope of 1. It can be observed a clear relation between the complexity reduction of the functionalities when refactored by the heuristic algorithm and by the expert, since the average reduction of both differs only by 7.3%, with the tool being able to achieve bigger complexity reductions on average. In the last column, we present the difference, in terms of complexity, of the best refactoring calculated by the tool and the complexity of the refactoring with the same orchestrator as chosen by the expert. As it can be seen in cells with a 0% distance value, 5 out of the 9 functionalities, the tool and the expert selected the same orchestrator. Note that the value is 0% because orchestration distance comparison is not done with the complexity values reported by the expert, but with the values calculated by our tool.

After having the best refactoring estimated by the algorithm, the next step of the evaluation involved manually verification in the source code, to check if the recommended orchestrations are feasible, for instance, that they do not break any real data dependence between entity accesses.

The selection process of the functionalities was based on the values from Table 2 and the functionalities selected fall in one of the following categories: (1) the refactoring tool achieved better complexity reduction; (2) the expert refactoring achieved better reduction; (3) the orchestrators selected by the tool and the expert were different; (4) the initial complexity value calculated by the tool is significantly different than the one calculated by the expert. This allows us to identify 6 cases where to do this analysis.

The refactoring of the *mergeCategories* functionality has a complexity reduction of 61.4% when compared to the initial architecture and converted 32 independent cluster invocations to 5. The algorithm chose a different orchestrator than the expert and achieved a complexity 17.2% lower. After manually checking the source code, we verified that this refactor

[‖]https://github.com/socialsoftware/edition.

10970024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 Functionality migration complexity reduction resulting from a refactoring using the tool with $\delta = 1$ and by an expert

	Initial FMC		Final FMC		Reduction %		
Functionality	**Tool**	**Expert**	**Tool**	**Expert**	**Tool**	**Expert**	**Orchestrator distance**
removeTweets	151	134	109	82	27.8%	38.8%	0%
getTaxonomy	317	317	159	192	49.8%	39.4%	1.24%
createLinearVE	2978	1790	263	383	91.1%	78.6%	0%
signup	1550	1490	314	376	79.7%	74.8%	0%
approveParticipant	193	190	113	147	41.5%	22.6%	0%
associateCategory	1813	1803	642	662	64.6%	63.3%	14.97%
deleteTaxonomy	261	253	187	164	28.4%	35.2%	11.79%
dissociate	806	772	358	489	55.6%	36.7%	0%
mergeCategories	485	453	187	253	61.4%	44.2%	37.87%
Averages	950.4	800.2	259.1.1	305.3	55.5%	48.2%	

is a valid option, all the operations are valid and the data dependencies that exist in the functionality are all taken into consideration. We can see that the tool computed fewer cluster invocations, fewer entity accesses, and fewer write operations, which reduces the complexity compared to the expert refactor. By looking at the code, we can see that the expert choice was based on selecting the cluster that contains more entity accesses as the orchestrator, which resulted in more repetitive invocations of other clusters. By choosing an orchestrator that was not so obvious at first glance, the algorithm was able to obtain a better refactor. The same happened for the functionality *getTaxonomy*, where the tool recommends a different orchestration than the expert, but it was able to obtain an higher reduction of the complexity by having all the operations condensed in 1 invocation per cluster, while the expert refactorization created more intermediate states. The source code also revealed that no data dependencies were broken by the tool's operations, and the sequence is applicable.

In the functionality *approveParticipant*, both, the tool and the expert, selected the same orchestrator cluster and did a very similar refactorization, where only 1 additional merge operation by the tool was enough to reduce the complexity in 41.5% when compared to the expert's 22.6%. The functionality *dissociate* falls in this same category since it shows that the tool was able to achieve a lower complexity with the same orchestrator, however by checking the source code we see that some invocations were merged while having a data dependence with previous invocations, which reveals invalid merges. This occurred because the data dependence distance had a scope > 1 and due to the test being executed with $\delta = 1$ the tool did not consider it when verifying if an invocation can be merged.

The refactor of *removeTweets* saw a complexity reduction of 27.8% when compared to the initial architecture and converted 14 independent cluster invocations to 7. Both, the algorithm and the expert, chose the same orchestrator and performed a very similar refactor, but the expert refactor achieves a higher complexity reduction. After manually checking the source code, we verified that this refactor is a valid option, except for one invocation that could have been merged with a previous one. Since in the original trace there was a data dependence between that local transaction and the one immediately before, the tool did not do the merge. This behavior reveals that the heuristic rules can be too restrictive and since the static analysis is not capturing the data-flow, we cannot accurately conclude that a write following a read does not use the data, which is something that an expert can directly observe by reading the code.

In functionality *createLinearVE* there is a significant difference of 1188 between the tool's initial complexity and the one calculated manually by the expert. This functionality is quite complex, with 108 local transactions and 210 entity accesses, and from looking at the source code, it involves many if/else conditions that define which accesses are performed during run-time. Ultimately, this difference relates to how the static analysis builds the functionality trace since it does not account for conditions and so all branches are collapsed in a single sequence. When the expert manually calculated the complexities, he only considered one of the if/else conditions, which resulted in fewer entity accesses and lower initial complexity.

To answer the first research question (RQ1) fully, we can say that most of the refactorizations applied to LdoD were feasible. However, there was at least 1 case where data dependencies were broken due to the configuration of the data dependence scope δ as 1. This difficulty when defining thresholds is still one of the major drawbacks of most heuristic techniques for code-smell and design-pattern detection.[29]

10970024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

6.3 | Orchestrators characterization

The validation of the refactorization of the LdoD codebase, when compared to the expert, has shown that the recommended refactorings highly reduce the complexity of migrating the functionalities and they, in most of the cases, even when the scope is 1, do not break the data dependencies between the accesses. Another interesting observation is that the heuristic can suggest better refactors than the ones envisioned by the expert. Trying to explain why the experts intuition may be misleading, take us to the second research question (RQ2): what are the characteristics of a cluster that make it a better fit for being a *SAGA* orchestrator.

We defined 4 metrics to measure, for each cluster, and in the context of the functionality being refactored, several aspects, like the number of read and write accesses to the cluster domain entities, the number of times the cluster is invoked in the context of the functionality, and the number of the invocations that cannot be merged due to a data dependence.

These metrics were visualized for the orchestrators of each one of the recommended refactorizations for 78 codebases and correlations were calculated between the value of the metrics and the reduction of the complexity. Table 3 shows the resulting correlations with values between $[-0.145, +0.210]$, which demonstrates that the data has a big variation and does not necessarily follow a trend line, with points that are too dispersed in space. The metric with higher correlation was related to the metric for reads accesses of orchestrators, but, even though, not significative.

The low correlation values show that the metrics cannot characterize the orchestrator, due to the great diversification of code patterns in the functionalities. A good orchestrator for a given codebase might have a lot of invocations, for example, while in another codebase we verified that the opposite may happen, which makes it hard to set a characterization for an orchestrator that applies to all the cases.

To answer the second research question (RQ2), we can conclude that it was not possible to extract enough metrics from the functionality sequence to characterize what is a good orchestrator. This increases the need of the refactoring heuristic tool which applies refactorization operations and validates if a given candidate orchestrator has the lowest complexity.

6.4 | Performance analysis

The performance analysis of the tool was done in the context of the large dataset of 78 codebases and considering 3 values for the data dependence threshold to evaluate how the tool behaves for different configurations. The set of machines used for the performance analysis are shown in Table 4 and consist of the local machine of one of the authors, and two Amazon Web Services EC2 dedicated instances that were started exclusively to test the tool, and did not share resources with other users of the cloud service.

The test results were extracted by refactoring all of the codebases in the dataset ten times for each value of the data dependence threshold in each one of the machines. Each measure is the result of averaging the same metric for all ten executions.

In Tables 5, 6 and 7 we present, for each machine and data dependence threshold, the average, lowest and highest duration, both, for refactoring a complete codebase and for refactoring a single functionality, in any codebase of the dataset.

TABLE 3 Correlation between the metrics for the orchestrator cluster and the reduction of the functionality migration complexity and system added complexity

	Correlation (r)		
Metric	**FMC**	**SAC**	**FMC+SAC**
Number of read accesses to domain entities	0.145	−0.083	0.014
Number of write accesses to domain entities	−0.145	0.083	−0.014
Number of times the cluster is invoked in functionality	−0.103	−0.090	−0.112
Number of initial data dependencies, when $\delta = \infty$	−0.110	0.210	0.087

1097024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 4 Machines used to test the refactorization tool

Machine	CPU		
	Model	Frequency	Cores
Local	Intel i5	2.90 GHz	4
AWS m5.xlarge	Intel Xeon Platinum 8000 (Skylake)	3.1 GHz	4
AWS c5.4xlarge	Intel Xeon Scalable Processors (Cascade Lake)	3.4 GHz	16

TABLE 5 Execution times when refactoring the dataset in the **Local** machine

Data dependence	Codebase refactor			Functionality refactor		
	Average	Lowest	Highest	Average	Lowest	Highest
$delta = 1$	4080 ms	1.40 ms	64,520 ms	4440 ms	1.38 ms	64,520 ms
$delta = 2$	3340 ms	1.38 ms	63,050 ms	3690 ms	1.35 ms	63,030 ms
$delta = \infty$	2790 ms	1.49 ms	68,430 ms	3060 ms	1.47 ms	68,320 ms

TABLE 6 Execution times when refactoring the dataset in the **AWS m5.xlarge** machine

Data dependence	Codebase refactor			Functionality refactor		
	Average	Lowest	Highest	Average	Lowest	Highest
$delta = 1$	2170 ms	1.05 ms	52,660 ms	2160 ms	1.03 ms	52,660 ms
$delta = 2$	2210 ms	0.98 ms	52,940 ms	2190 ms	0.96 ms	52,940 ms
$delta = \infty$	2140 ms	1.05 ms	52.84 s	2130 ms	1.03 ms	52,780 ms

TABLE 7 Execution times when refactoring the dataset in the **AWS c5.4xlarge** machine

Data dependence	Codebase refactor			Functionality refactor		
	Average	Lowest	Highest	Average	Lowest	Highest
$delta = 1$	1100 ms	0.86 ms	21,910 ms	1040 ms	0.84 ms	21,910 ms
$delta = 2$	1100 ms	0.86 ms	21,800 ms	1050 ms	0.84 ms	21,800 ms
$delta = \infty$	1060 ms	0.86 ms	21,670 ms	1020 ms	0.84 ms	21,670 ms

As expected, the value of $delta = 1$ results in higher refactorization times since this is the case where the algorithm has more freedom to execute sequence change operations. Since fewer data dependencies are considered, this results in more recursive iterations of the merge of invocations. On the other hand, the value $delta = \infty$ results in the lowest execution times both for refactoring a complete codebase and for refactoring a single functionality, since, with this threshold value, the tool will consider more data dependencies and will not be able to execute as many sequence change operations as it would with another value.

By analysing the results, we can observe that there is a relation between the number of CPU cores in the machine and a lower refactorization duration for each codebase and its functionalities. This is a very CPU intensive application that requires a lot of logical computations, so a more powerful CPU like the one in the AWS c5.4xlarge machine will ultimately reduce the execution time.

When it comes to the parallelism of the tool, it is interesting to note that the codebase that took the longest to refactor is the one that contains the functionality that also took the longest to refactor. This behavior clearly shows the parallelism of the application, since while the most extensive functionality is being refactored, all the other ones have finishing their refactoring. Therefore, the total amount of time that the tool takes to compute all the *SAGA*s in a codebase is equal to the time it takes to compute it for the heaviest functionality.

1097024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

6.5 | Summary

In summary, the analysis shows that:

- There is a very positive impact on the complexity reduction when analyzing the refactorings made by the service;
- The service is able to reduce both the number of external requests between microservices and the number of domain entities access;
- The impact of changing the scope of data dependencies is not significant, since in most of the functionalities there is a data dependence between a local transaction and the local transaction immediately before, so increasing the value won't change the proposed refactorization;
- Most of the refactorizations analysed in depth are feasible, however we have seen cases where a wrong configuration of the data dependence scope resulted in breaking changes;
- We were not able to determine metrics that can characterize possible good orchestrators in the monolithic version of the controllers;
- The performance analysis of the tool for a large codebase showed that the algorithm is very efficient when it comes to the time it takes to refactorize a codebase.

An overall analysis, lessons learned, provide the following feedback for researchers:

- It is possible to obtain reliable refactorings using a heuristic approach, without requiring the learning phase of machine learning approaches;
- The difficulty of applying a machine leaning approach to this problem is related to the difficult of identifying metrics (patterns) that characterize good orchestrators.

On the other hand, practitioners can also benefit from the results:

- In same cases the architect intuitions shown to result on worse refactorizations, so, the use of the tool will help the developer in the refactoring activities;
- The complexity reduction achieved through the recommended refactorings is significant, which shows the applicability of the approach and the importance of refactoring fine-grained invocations into coarse-grained ones.

6.6 | Threats to validity

In terms of internal validity, it is verified that the refactorings provide a significant reduction of the complexity, even though the data dependencies are inferred from the analysis of the sequence of accesses and not directly obtained from the code. This may have impact on the accuracy of the results, but the experiment with variations of the scope of data dependencies has shown consistent results. Therefore, even if the architect uses the wider scope, the recommended refactorings do not have a significant variation on the complexity. The sequences of accesses used for the validation, obtained from static analysis linearize all the accesses of a functionality in a single sequence. However, we obtained results similar to the expert refactorizations, sometimes even better. This also depends on the used dataset, a future version of that static analyser that captures the data-flow will allow to work with a dataset that will provide more precise results.

In terms of external validity, the dataset is for a small set of technologies, Spring-Boot and JPA, but the functionalities logic is technology independent. On the other hand, the approach is generalizable to any context where we have functionality traces on domain entities, as described by the formalization. The implementation was done in Mono2Micro and the traces were generated from the call graphs obtained through static analysis, which also serves as a proof of concept.

7 | CONCLUSIONS

This paper proposes a heuristic to help a software architect on the identification of the possible refactorings of monolith functionalities, to reduce their migration effort to a microservices architecture applying the *SAGA* pattern.

1097024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

This effort is due to the functionality migration, because of the introduction of eventual consistency on the functionality behavior, which adds an extra complexity to its implementation.

The results show that the tool was able to recommend refactorings that allow a significant reduction of the functionalities migration complexity. Interestingly, some of the recommended refactorings provide higher reductions than the ones identified by an expert, which suggests that it would be interesting to characterize what are the properties of a good candidate for a *SAGA* orchestrator. However, a study with 4 metrics that characterize the clusters accessed by the functionality, shows that there is no statistical significance in the correlation between the values of the metrics and the reductions on complexity. This problem is left for future work, and also indicates that a heuristic approach, which calculates all combinations, fits these cases where there is significant variation in the dataset.

Overall, the refactoring tool can be useful on the identification of how the monolith functionalities can be refactored such that their migration to a microservices architecture have the lowest possible complexity. Therefore, the tool has two benefits, given a functionality and a decomposition: (1) it identifies the refactorings that have lower complexity; (2) it shows how the sequences of functionality accesses to the domain entities are changed by the refactoring. Due to the first advantage, it recommends to the architect what should be the *SAGA* orchestrator, while the second shows which refactoring operations should be done on the functionality.

ACKNOWLEDGMENTS

This work was supported by national funds through FCT, Fundação para a Ciência e a Tecnologia, under project UIDB/50021/2020.

AUTHOR CONTRIBUTIONS

José Correia: Conceptualization, Methodology, Tool Development, Validation, Writing Article; **António Rito Silva:** Conceptualization, Methodology, Writing Article, Supervision.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are openly available in `mono2micro` at https://github.com/socialsoftware/mono2micro.

ORCID

António Rito Silva https://orcid.org/0000-0001-9840-457X

REFERENCES

1. O'Hanlon C. A Conversation with Werner Vogels. *Queue*. 2006;4(4):14-22. doi:10.1145/1142055.1142065
2. Fowler M. Microservices. http://martinfowler.com/articles/microservices.html.
3. Gysel M, Kölbener L, Giersche W, Zimmermann O. Service cutter: A systematic approach to service decomposition. Proceedings of the European Conference on Service-Oriented and Cloud Computing, Springer; 2016: 185–200. doi: 10.1007/978-3-319-44482-612
4. Mazlami G, Cito J, Leitner P. Extraction of microservices from monolithic software architectures. Proceedings of the 2017 IEEE International Conference on Web Services (ICWS); 2017: 524–531. doi: 10.1109/ICWS.2017.61
5. Nakazawa R, Ueda T, Enoki M, Horii H. Visualization tool for designing microservices with the monolith-first approach. Proceedings of the 2017 IEEE International Conference on Web Services (ICWS); 2018: 32–42. doi: 10.1109/VISSOFT.2018.00012
6. Jin W, Liu T, Cai Y, Kazman R, Mo R, Zheng Q. Service Candidate Identification from Monolithic Systems Based on Execution Traces. *IEEE Trans Software Eng*. 2021;47(5):987-1007. doi:10.1109/TSE.2019.2910531
7. Nunes L, Santos N, Silva AR. From a monolith to a microservices architecture: An approach based on transactional contexts. Proceedings of the European Conference on Software Architecture, Springer; 2019: 37–52. doi: 10.1109/TSE.2019.2910531
8. Gomes Barbosa MH, Maia PHM Towards identifying microservice candidates from business rules implemented in stored procedures. Proceedings of the 2020 IEEE International Conference on Software Architecture Companion (ICSA-C); 2020: 41–48. doi: 10.1109/ICSA-C50368.2020.00015
9. Daoud M, El Mezouari A, Faci N, Benslimane D, Maamar Z, El Fazziki A. Automatic microservices identification from a set of business processes. Proceedings of the International Conference on Smart Applications and Data Analysis, Springer; 2020: 299–315. doi: 10.1007/978-3-030-45183-723
10. Selmadji A, Seriai AD, Bouziane HL, Oumarou Mahamane R, Zaragoza P, Dony C. From monolithic architecture style to microservice one based on a semi-automatic approach. Proceedings of the 2020 IEEE International Conference on Software Architecture (ICSA); 2020: 157–168. doi: 10.1109/ICSA47634.2020.00023
11. Almeida JF, Silva AR. Monolith migration complexity tuning through the application of microservices patterns. Proceedings of the European Conference on Software Architecture, Springer; 2020: 39–54. doi: 10.1007/978-3-030-58923-33

1097024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

12. Brewer EA. Towards robust distributed systems (Abstract). Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing PODC'00. Association for Computing Machinery, New York; 2000: 7. doi: 10.1145/343477.343502
13. Haerder T, Reuter A. Principles of Transaction-Oriented Database Recovery. *ACM Comput Surv*. 1983;15:287-317.
14. Bailis P, Ghodsi A. Eventual consistency today: Limitations, extensions, and beyond. *Commun ACM*. 2013;56(5):55-63.
15. Richardson C. *Microservices Patterns: With Examples in Java*. Manning Publications; 2019.
16. Zanoni M, Fontana FA, Stella F. On applying machine learning techniques for design pattern detection. *J Syst Software*. 2015;103:102-117. doi:10.1016/j.jss.2015.01.037
17. Khomh F, Vaucher S, Guéhéneuc YG, Sahraoui H. A Bayesian approach for the detection of code and design smells. Proceedings of the 2009 Ninth International Conference on Quality Software; 2009: 305–314. doi: 10.1109/QSIC.2009.47
18. Garcia-Molina H, Salem K. Sagas. Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data SIGMOD'87. Association for Computing Machinery, New York; 1987: 249–259. doi: 10.1145/38713.38742
19. Santos N, Rito SA. A complexity metric for microservices architecture migration. Proceedings of the 2020 IEEE International Conference on Software Architecture (ICSA); 2020: 169–178. doi: 10.1109/ICSA47634.2020.00024
20. Andrade B, Santos S, Silva AR. From monolith to microservices: static and dynamic analysis comparison. 2022. doi:10.48550/ARXIV.2204.11844
21. Fritzsch J, Bogner J, Zimmermann A, Wagner S. From monolith to microservices: a classification of refactoring approaches. Proceedings of the International Workshop on Software Engineering Aspects of Continuous Development and New Paradigms of Software Production and Deployment, Springer; 2018: 128–141. doi: 10.1007/978-3-030-06019-010
22. Abdellatif M, Shatnawi A, Mili H, et al. A taxonomy of service identification approaches for legacy software systems modernization. *J Syst Software* 2021; 173: 110868. doi: 10.1016/j.jss.2020.110868
23. Hassan S, Bahsoon R, Minku L, Ali N. Dynamic Evaluation of Microservice Granularity Adaptation. *ACM Trans Autonom Adapt Syst*. 2022;16(2):1-35. doi:10.1145/3502724
24. Bogner J, Wagner S, Zimmermann A. Automatically measuring the maintainability of service- and microservice-based systems: a literature review. *Proceedings of the 27th International Workshop on Software Measurement and 12th International Conference on Software Process and Product Measurement* IWSM Mensura'17. Association for Computing Machinery, New York; 2017: 107–115. doi: 10.1145/3143434.3143443
25. Hassan S, Bahsoon R, Buyya R. Systematic scalability analysis for microservices granularity adaptation design decisions. *Software Pract Exp*. 2022;52:1378-1401. doi:10.1002/spe.3069
26. Palomba F, Bavota G, Penta MD, Oliveto R, Poshyvanyk D, Lucia AD. Mining version histories for detecting code smells. *IEEE Trans Software Eng*. 2010;36:20-36. doi:10.1109/TSE.2009.50
27. Moha N, Gueheneuc YG, Duchien L, Meur AFL. A method for the specification and detection of code and design smells. *IEEE Trans Software Eng*. 2015;41:462-489. doi:10.1109/TSE.2014.2372760
28. Palomba F, Panichella A, De Lucia A, Oliveto R, Zaidman A. A textual-based technique for smell detection. Proceedings of the 2016 IEEE 24th International Conference on Program Comprehension (ICPC); 2016: 1–10. doi: 10.1109/ICPC.2016.7503704
29. Pecorelli F, Palomba F, Nucci DD, Lucia AD. Comparing heuristic and machine learning approaches for metric-based code smell detection. ICPC'19: Proceedings of the 27th International Conference on Program Comprehension. 2019; 11681: 93–104. doi: 10.1109/ICPC.2019.00023
30. Zanoni M, Fontana FA, Stella F. Comparing and experimenting machine learning techniques for code smell detection. *Emp Software Eng*. 2016;21:1143-1191. doi:10.1007/s10664-015-9378-4
31. Zanoni M, Fontana FA. Code smell severity classification using machine learning techniques. *Knowl Based Syst*. 2017;128:43-58. doi:10.1016/j.knosys.2017.04.014
32. Ma S, Chuang Y, Lan C, Chen H, Huang C, Li C. Scenario-based microservice retrieval using Word2Vec. Proceedings of the 2018 IEEE 15th International Conference on e-Business Engineering (ICEBE); 2018: 239–244. doi: 10.1109/ICEBE.2018.00046
33. Abdullah M, Iqbal W, Erradi A. Unsupervised learning approach for web application auto-decomposition into microservices. *J Syst Software*. 2019;151:243-257. doi:10.1016/j.jss.2019.02.031

How to cite this article: Correia J, Rito Silva A. Identification of monolith functionality refactorings for microservices migration. *Softw Pract Exper*. 2022;52(12):2664-2683. doi: 10.1002/spe.3141

1097024x, 2022, 12, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/spe.3141 by Ned Univ of Engineering & Tech, Wiley Online Library on [10/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License